

Self-Referential Structures with Pointers (Self-Learning)

Q. Explain the concept of Structures in C. How can we use pointers with structures? Provide a code example to demonstrate its use.

> **Definition to Remember**

1. Structures in C

- Groups multiple variables (called **members/fields**) of different types into one logical unit.

2. Pointers with Structures

When a structure is passed to a function **by value**, a full copy is made wasteful for large structures. Passing a **pointer** to the structure is more efficient.

- **Arrow Operator** (`->`) is used to access members via a pointer.

3. Self-Referential Structure

A structure that contains a **pointer to another structure of the same type** the building block of **Linked Lists and Trees**.

4. Complete Code Example

```
```c
```

```
struct Student {
```

```
int main() {

 // Dynamic Memory Allocation

 if (ptr == NULL) {

 printf("Memory not allocated\n");

 return 1;
 }

 // Access members using Arrow Operator (->)

 ptr->name = "Suraj";

 printf("Student Name: %s\n", ptr->name);

 free(ptr); // Release allocated memory

 return 0;
}
```

## 5. Dot vs Arrow Operator

| Situation | Operator | Syntax |

```

> **Must-Write Points (for 10 marks)**

> 1. `s`
```

```

> **Quick Recall**

> `stru`
to rele
```

## C Code: Self-Referential Structure Node Creation & Linked List Traversal

```
```c  
#include <stdio.h>  
#include <stdlib.h>
```

```
// Definition of Self-Referential Structure
```

```
struct l
```

```
// Function to traverse dynamic linked list
```

```
void pr
```

```
int main() {
```

```
// Dyna
```

```
// Assigning data & linking nodes
```

```
head->
```

```
second->data = 20;
```

```
secon
```

```
third->data = 30;
```

```
third->
```

```
printList(head);
```

```
// Free allocated memory
```

```
free(h
```

```
return 0;
```

```
}
```